Haylee Paredes

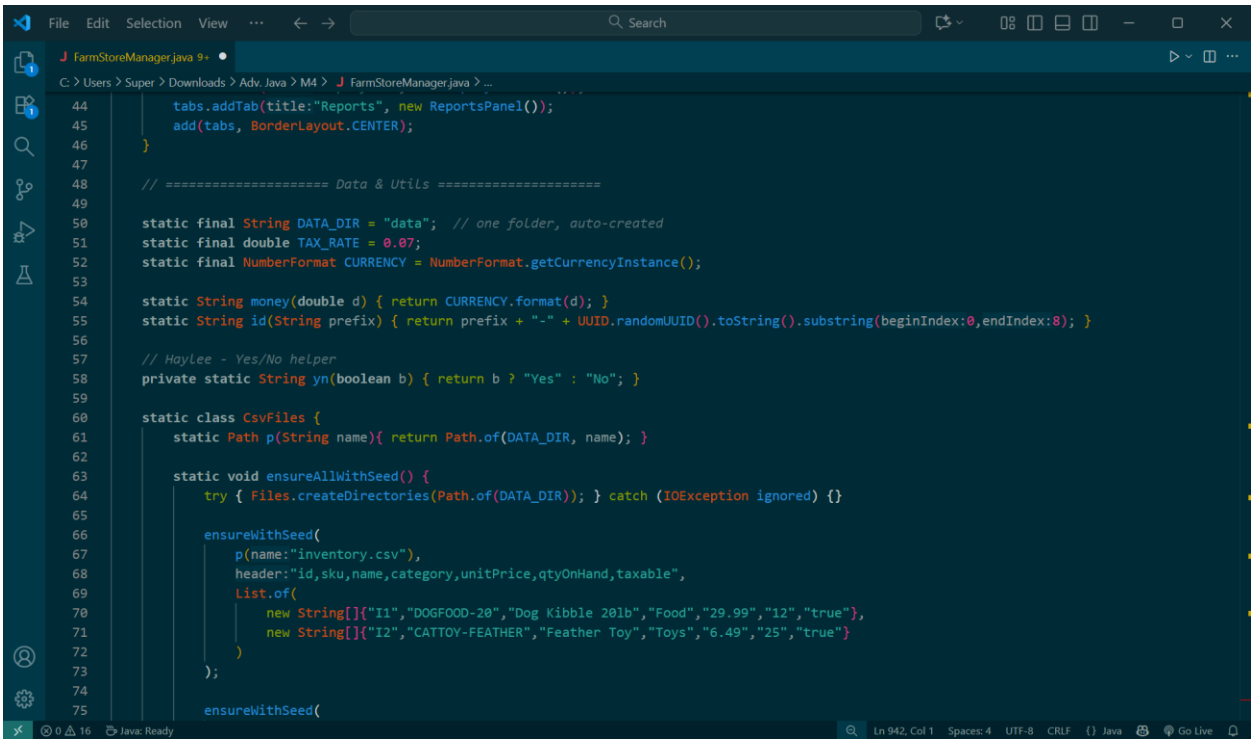
Oct. 30, 2025

CSC-251-0901

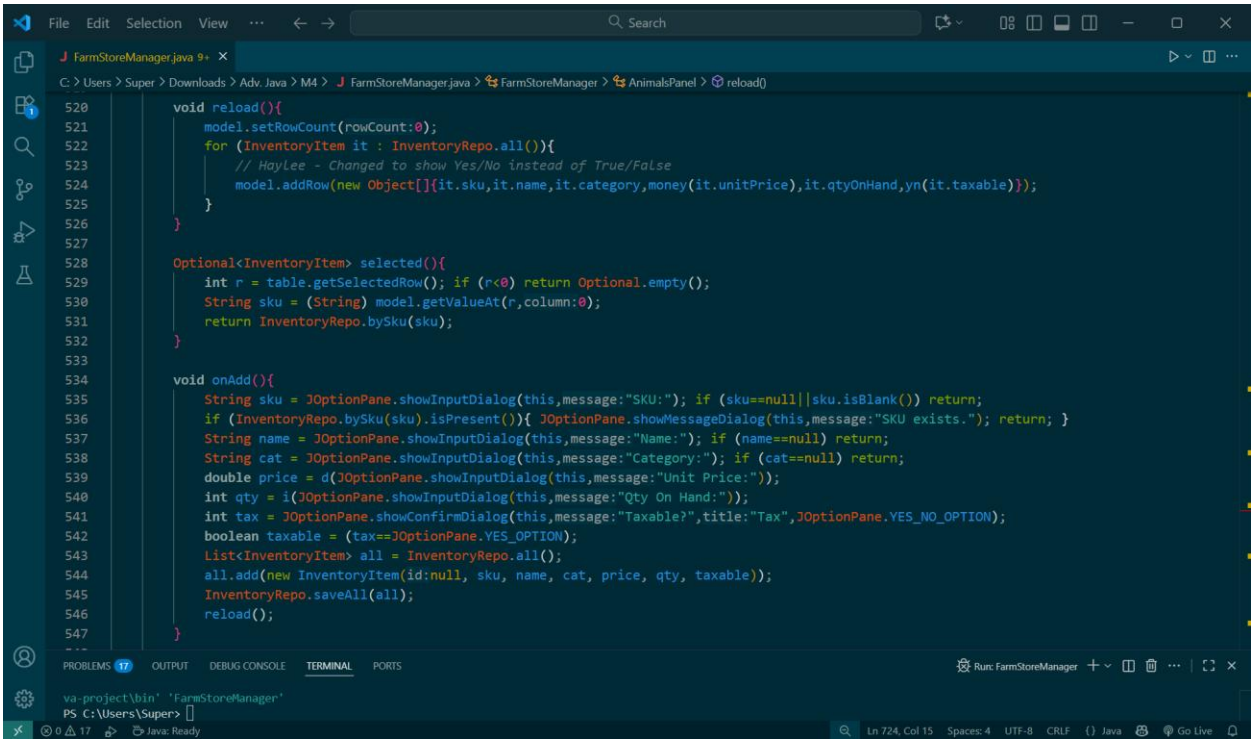
Advanced Java Programming

Lines 58, 524, and 725 are exchanging True/False for Yes/No.

Line 58:

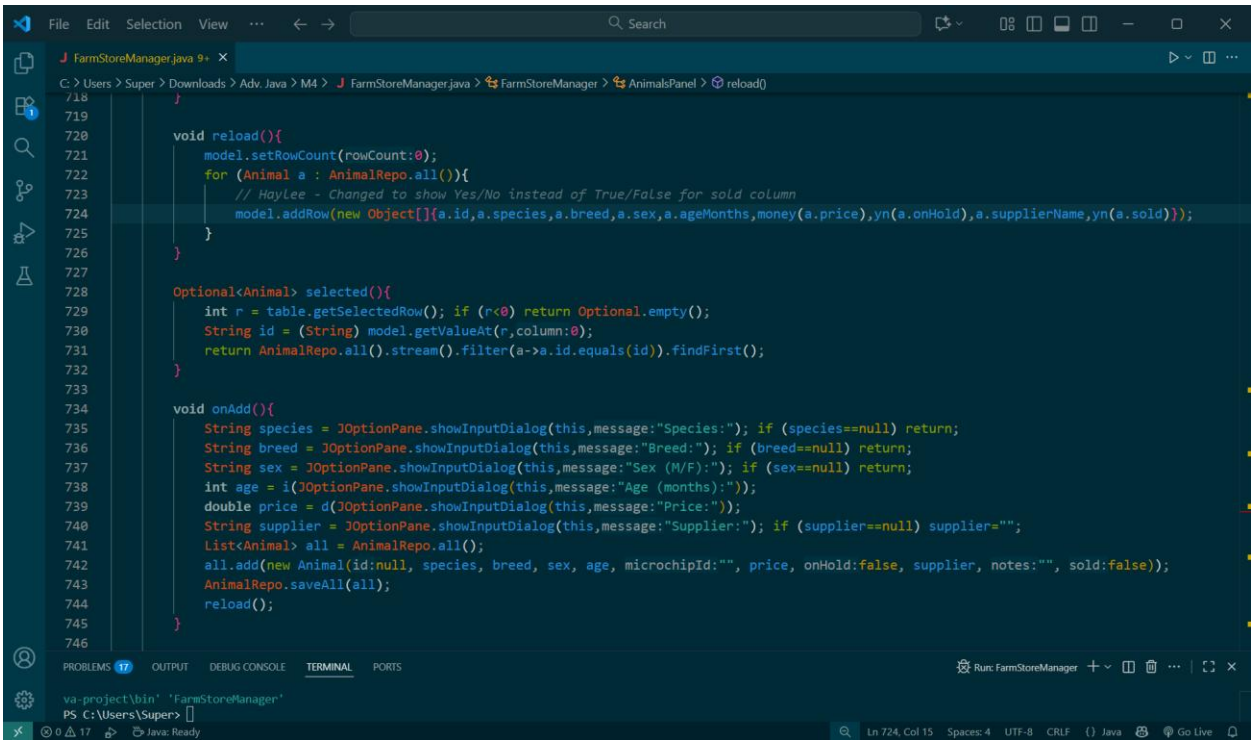


Line 524:




```
520         void reload(){
521             model.setRowCount(rowCount:0);
522             for (InventoryItem it : InventoryRepo.all()){
523                 // Haylee - Changed to show Yes/No instead of True/False
524                 model.addRow(new Object[]{it.sku,it.name,it.category,money(it.unitPrice),it.qtyOnHand,yn(it.taxable)});
525             }
526         }
527
528         Optional<InventoryItem> selected(){
529             int r = table.getSelectedRow(); if (r<0) return Optional.empty();
530             String sku = (String) model.getValueAt(r,column:0);
531             return InventoryRepo.bySku(sku);
532         }
533
534         void onAdd(){
535             String sku = JOptionPane.showInputDialog(this,message:"SKU:"); if (sku==null||sku.isBlank()) return;
536             if (InventoryRepo.bySku(sku).isPresent()){ JOptionPane.showMessageDialog(this,message:"SKU exists."); return; }
537             String name = JOptionPane.showInputDialog(this,message:"Name:"); if (name==null) return;
538             String cat = JOptionPane.showInputDialog(this,message:"Category:"); if (cat==null) return;
539             double price = d(JOptionPane.showInputDialog(this,message:"Unit Price:"));
540             int qty = i(JOptionPane.showInputDialog(this,message:"Qty On Hand:"));
541             int tax = JOptionPane.showConfirmDialog(this,message:"Taxable?",title:"Tax",JOptionPane.YES_NO_OPTION);
542             boolean taxable = (tax==JOptionPane.YES_OPTION);
543             List<InventoryItem> all = InventoryRepo.all();
544             all.add(new InventoryItem(id:null, sku, name, cat, price, qty, taxable));
545             InventoryRepo.saveAll(all);
546             reload();
547         }
```

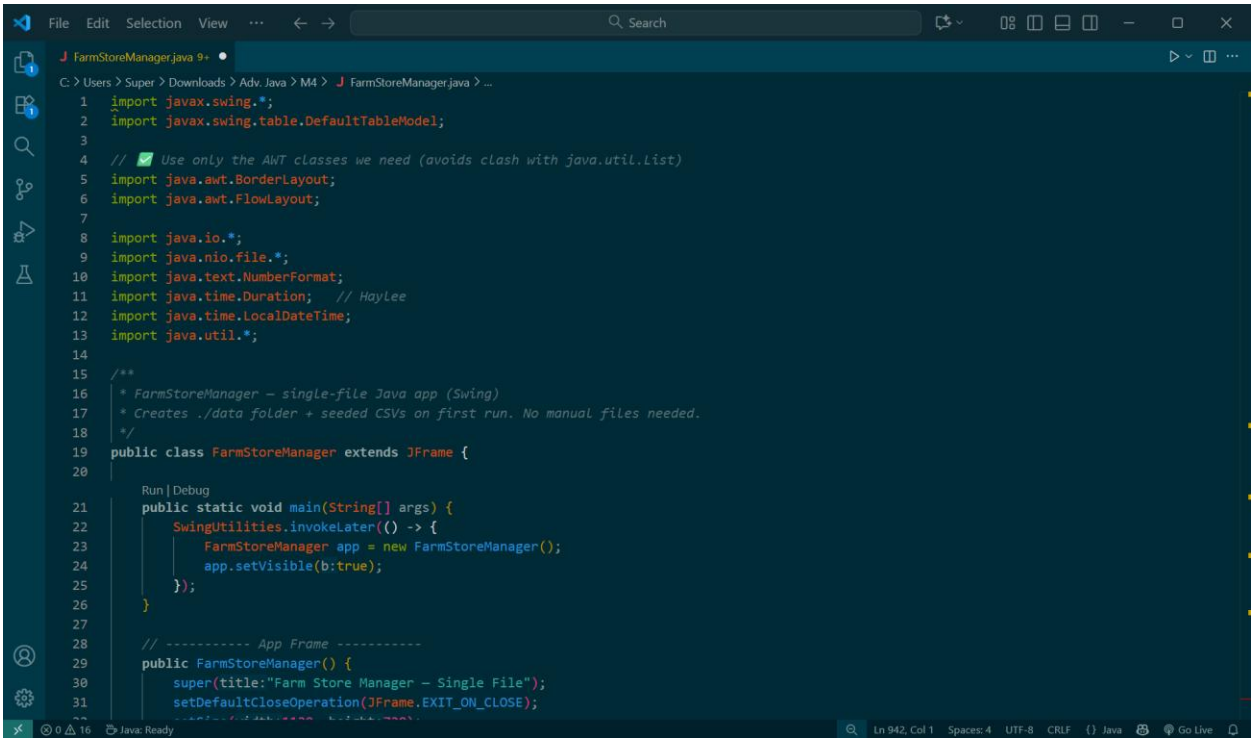
Line 724:




```
718         }
719
720         void reload(){
721             model.setRowCount(rowCount:0);
722             for (Animal a : AnimalRepo.all()){
723                 // Haylee - Changed to show Yes/No instead of True/False for sold column
724                 model.addRow(new Object[]{a.id,a.species,a.breed,a.sex,a.ageMonths,money(a.price),yn(a.onHold),a.supplierName,yn(a.sold)});
725             }
726         }
727
728         Optional<Animal> selected(){
729             int r = table.getSelectedRow(); if (r<0) return Optional.empty();
730             String id = (String) model.getValueAt(r,column:0);
731             return AnimalRepo.all().stream().filter(a->a.id.equals(id)).findFirst();
732         }
733
734         void onAdd(){
735             String species = JOptionPane.showInputDialog(this,message:"Species:"); if (species==null) return;
736             String breed = JOptionPane.showInputDialog(this,message:"Breed:"); if (breed==null) return;
737             String sex = JOptionPane.showInputDialog(this,message:"Sex (M/F):"); if (sex==null) return;
738             int age = i(JOptionPane.showInputDialog(this,message:"Age (months):"));
739             double price = d(JOptionPane.showInputDialog(this,message:"Price:"));
740             String supplier = JOptionPane.showInputDialog(this,message:"Supplier:"); if (supplier==null) supplier="";
741             List<Animal> all = AnimalRepo.all();
742             all.add(new Animal(id:null, species, breed, sex, age, microchipId:"", price, onHold:false, supplier, notes:"", sold:false));
743             AnimalRepo.saveAll(all);
744             reload();
745         }
746
```

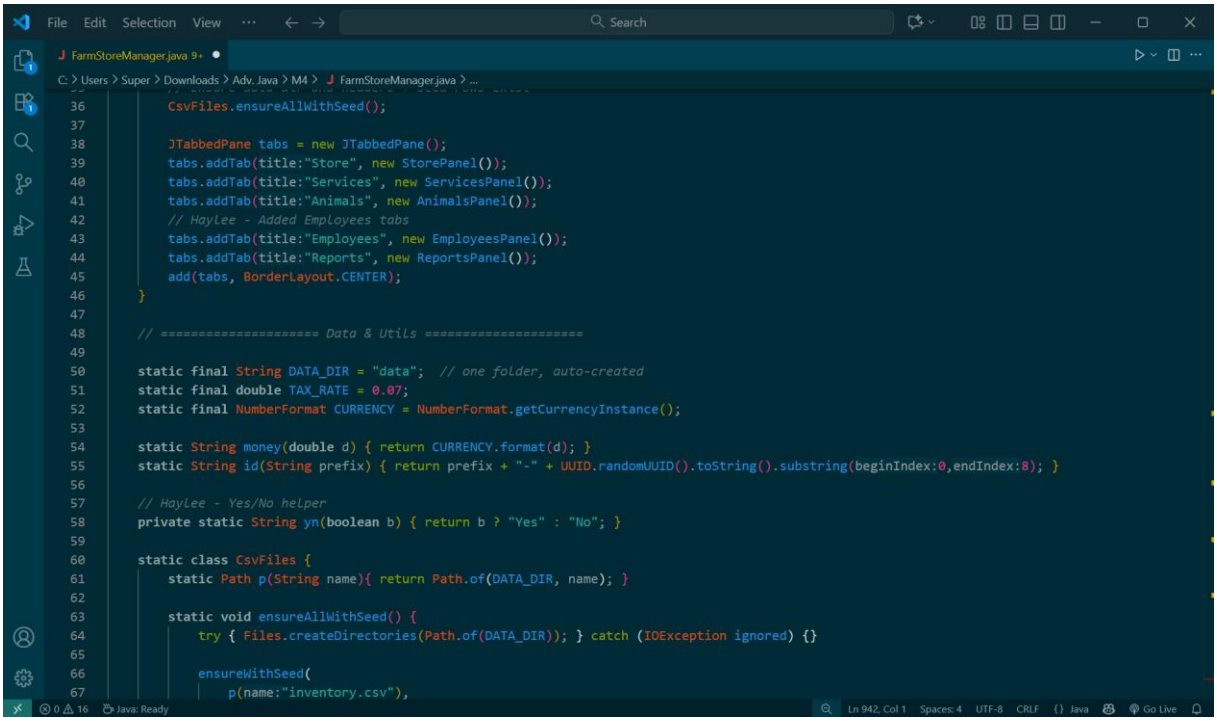
Lines 11, 43, 106-114, 249-272, 419-475, 784-910, and 934-937 are adding an employee's section to track clock ins and clock outs, and their paychecks.

Line 11:



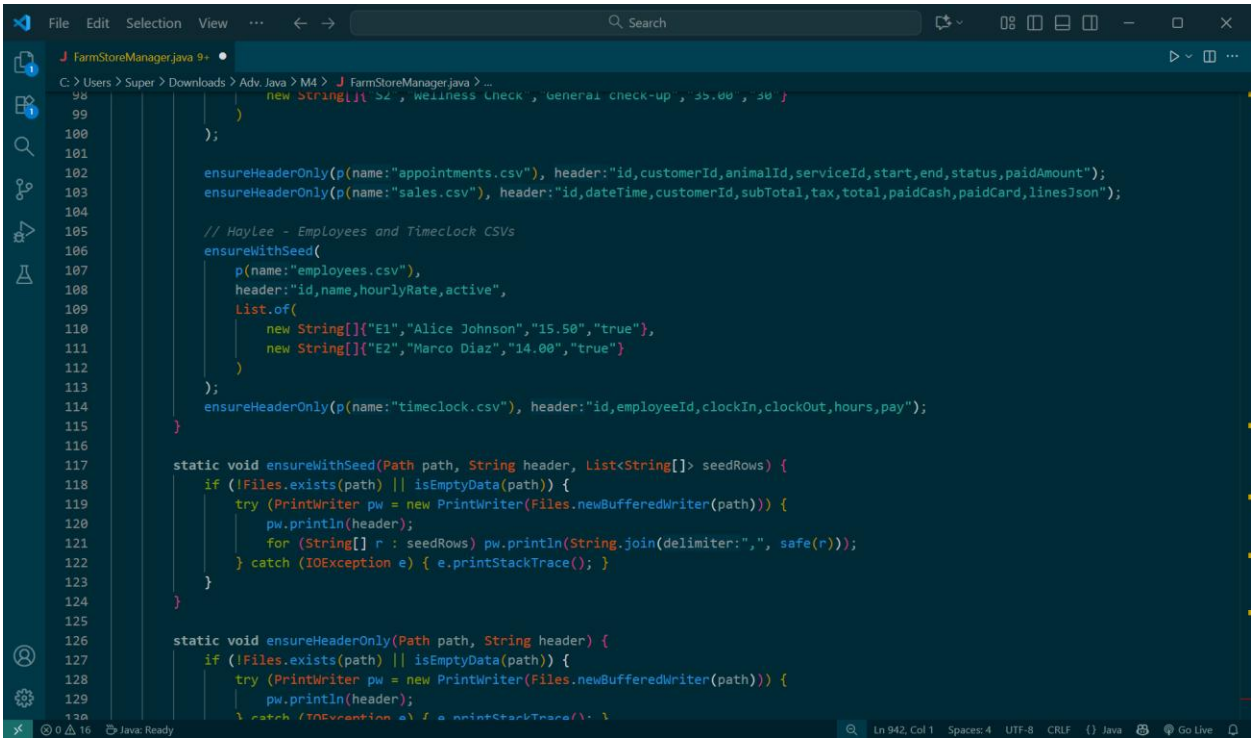
```
1  import javax.swing.*;
2  import javax.swing.table.DefaultTableModel;
3
4  // ✅ Use only the AWT classes we need (avoids clash with java.util.List)
5  import java.awt.BorderLayout;
6  import java.awt.FlowLayout;
7
8  import java.io.*;
9  import java.nio.file.*;
10 import java.text.NumberFormat;
11 import java.time.Duration;   // Haylee
12 import java.time.LocalDateTime;
13 import java.util.*;
14
15 /**
16  * FarmStoreManager — single-file Java app (Swing)
17  * Creates ./data folder + seeded CSVs on first run. No manual files needed.
18  */
19 public class FarmStoreManager extends JFrame {
20
       Run | Debug
21     public static void main(String[] args) {
22         SwingUtilities.invokeLater(() -> {
23             FarmStoreManager app = new FarmStoreManager();
24             app.setVisible(b:true);
25         });
26     }
27
28     // ---------- App Frame ----------
29     public FarmStoreManager() {
30         super(title:"Farm Store Manager — Single File");
31         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

Line 43:



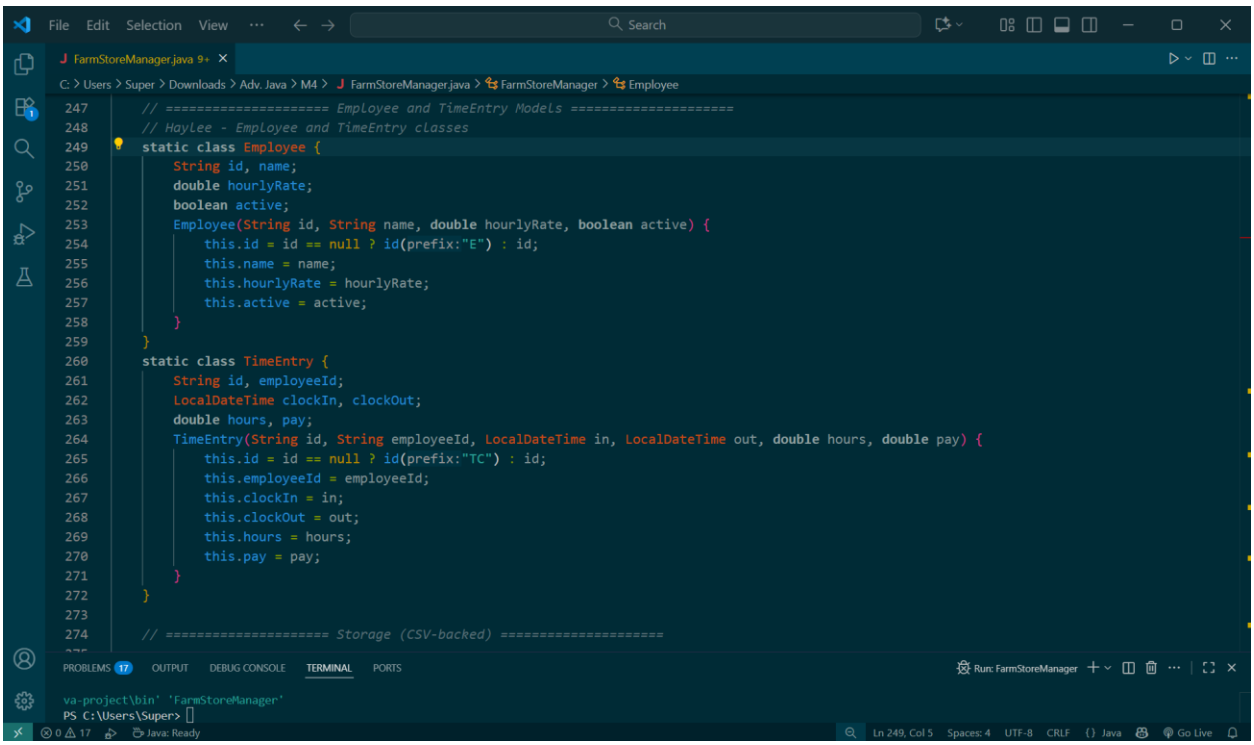
```
36         CsvFiles.ensureAllWithSeed();
37
38         JTabbedPane tabs = new JTabbedPane();
39         tabs.addTab(title:"Store", new StorePanel());
40         tabs.addTab(title:"Services", new ServicesPanel());
41         tabs.addTab(title:"Animals", new AnimalsPanel());
42         // Haylee - Added Employees tabs
43         tabs.addTab(title:"Employees", new EmployeesPanel());
44         tabs.addTab(title:"Reports", new ReportsPanel());
45         add(tabs, BorderLayout.CENTER);
46     }
47
48     // ==================== Data & Utils ====================
49
50     static final String DATA_DIR = "data";  // one folder, auto-created
51     static final double TAX_RATE = 0.07;
52     static final NumberFormat CURRENCY = NumberFormat.getCurrencyInstance();
53
54     static String money(double d) { return CURRENCY.format(d); }
55     static String id(String prefix) { return prefix + "-" + UUID.randomUUID().toString().substring(beginIndex:0,endIndex:8); }
56
57     // Haylee - Yes/No helper
58     private static String yn(boolean b) { return b ? "Yes" : "No"; }
59
60     static class CsvFiles {
61         static Path p(String name){ return Path.of(DATA_DIR, name); }
62
63         static void ensureAllWithSeed() {
64             try { Files.createDirectories(Path.of(DATA_DIR)); } catch (IOException ignored) {}
65
66             ensureWithSeed(
67                 p(name:"inventory.csv"),
```

Lines 106-114:



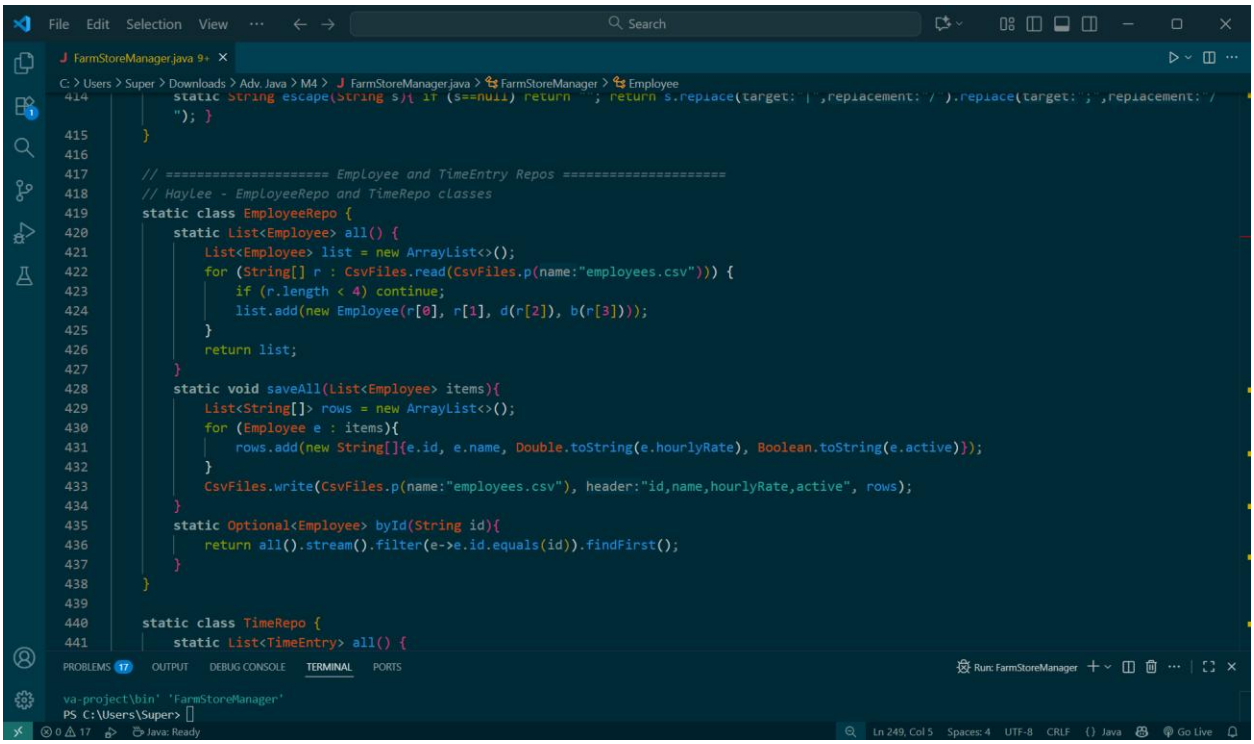
```
        ensureHeaderOnly(p(name:"appointments.csv"), header:"id,customerId,animalId,serviceId,start,end,status,paidAmount");
        ensureHeaderOnly(p(name:"sales.csv"), header:"id,dateTime,customerId,subTotal,tax,total,paidCash,paidCard,linesJson");

        // Haylee - Employees and Timeclock CSVs
        ensureWithSeed(
            p(name:"employees.csv"),
            header:"id,name,hourlyRate,active",
            List.of(
                new String[]{"E1","Alice Johnson","15.50","true"},
                new String[]{"E2","Marco Diaz","14.00","true"}
            )
        );
        ensureHeaderOnly(p(name:"timeclock.csv"), header:"id,employeeId,clockIn,clockOut,hours,pay");
    }

    static void ensureWithSeed(Path path, String header, List<String[]> seedRows) {
        if (!Files.exists(path) || isEmptyData(path)) {
            try (PrintWriter pw = new PrintWriter(Files.newBufferedWriter(path))) {
                pw.println(header);
                for (String[] r : seedRows) pw.println(String.join(delimiter:",", safe(r)));
            } catch (IOException e) { e.printStackTrace(); }
        }
    }

    static void ensureHeaderOnly(Path path, String header) {
        if (!Files.exists(path) || isEmptyData(path)) {
            try (PrintWriter pw = new PrintWriter(Files.newBufferedWriter(path))) {
                pw.println(header);
```

Lines 249-272:



```
// ==================== Employee and TimeEntry Models ====================
// Haylee - Employee and TimeEntry classes
static class Employee {
    String id, name;
    double hourlyRate;
    boolean active;
    Employee(String id, String name, double hourlyRate, boolean active) {
        this.id = id == null ? id(prefix:"E") : id;
        this.name = name;
        this.hourlyRate = hourlyRate;
        this.active = active;
    }
}
static class TimeEntry {
    String id, employeeId;
    LocalDateTime clockIn, clockOut;
    double hours, pay;
    TimeEntry(String id, String employeeId, LocalDateTime in, LocalDateTime out, double hours, double pay) {
        this.id = id == null ? id(prefix:"TC") : id;
        this.employeeId = employeeId;
        this.clockIn = in;
        this.clockOut = out;
        this.hours = hours;
        this.pay = pay;
    }
}

// ==================== Storage (CSV-backed) ====================
```

Lines 419-475:

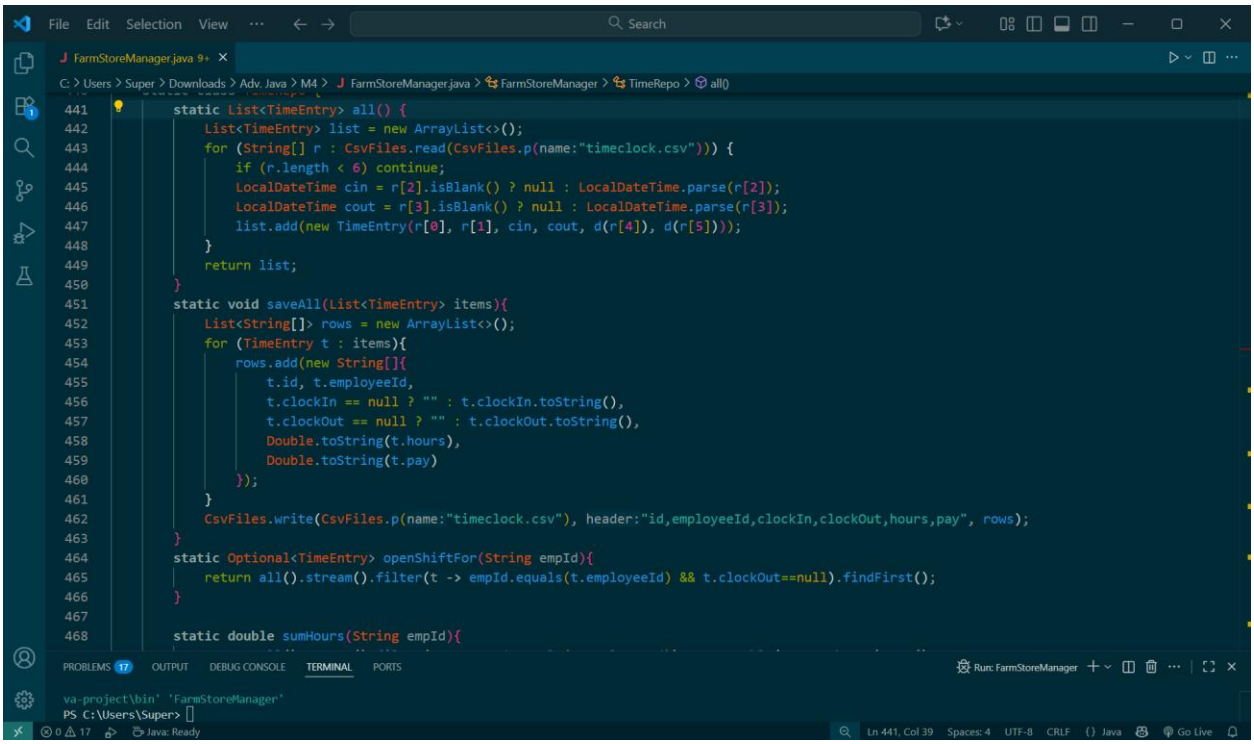


```
static String escape(String s){ if (s==null) return ""; return s.replace("|","/").replace(";","/");
"); }
}

// ==================== Employee and TimeEntry Repos ====================
// HayLee - EmployeeRepo and TimeRepo classes
static class EmployeeRepo {
    static List<Employee> all() {
        List<Employee> list = new ArrayList<>();
        for (String[] r : CsvFiles.read(CsvFiles.p("employees.csv"))) {
            if (r.length < 4) continue;
            list.add(new Employee(r[0], r[1], d(r[2]), b(r[3])));
        }
        return list;
    }
    static void saveAll(List<Employee> items){
        List<String[]> rows = new ArrayList<>();
        for (Employee e : items){
            rows.add(new String[]{e.id, e.name, Double.toString(e.hourlyRate), Boolean.toString(e.active)});
        }
        CsvFiles.write(CsvFiles.p("employees.csv"), "id,name,hourlyRate,active", rows);
    }
    static Optional<Employee> byId(String id){
        return all().stream().filter(e->e.id.equals(id)).findFirst();
    }
}

static class TimeRepo {
    static List<TimeEntry> all() {
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Run: FarmStoreManager

```
va-project\bin' 'FarmStoreManager'
PS C:\Users\Super>
```

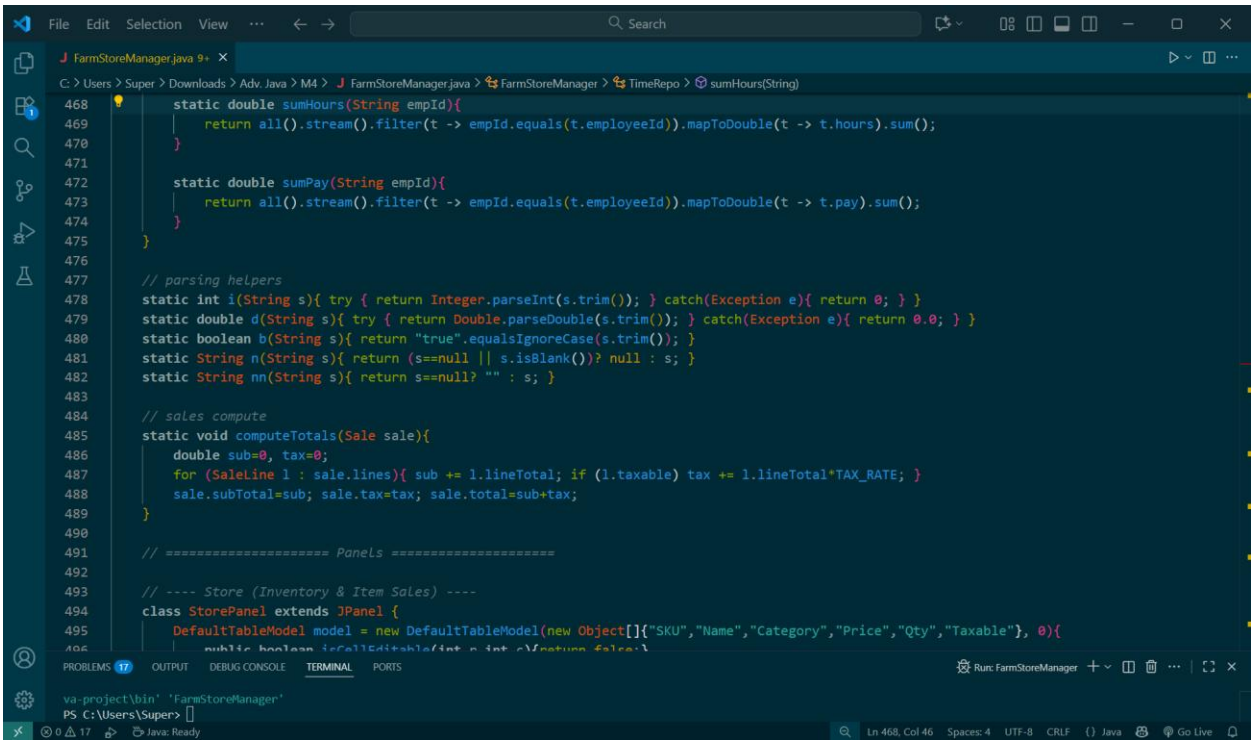


```
    static List<TimeEntry> all() {
        List<TimeEntry> list = new ArrayList<>();
        for (String[] r : CsvFiles.read(CsvFiles.p("timeclock.csv"))) {
            if (r.length < 6) continue;
            LocalDateTime cin = r[2].isBlank() ? null : LocalDateTime.parse(r[2]);
            LocalDateTime cout = r[3].isBlank() ? null : LocalDateTime.parse(r[3]);
            list.add(new TimeEntry(r[0], r[1], cin, cout, d(r[4]), d(r[5])));
        }
        return list;
    }
    static void saveAll(List<TimeEntry> items){
        List<String[]> rows = new ArrayList<>();
        for (TimeEntry t : items){
            rows.add(new String[]{
                t.id, t.employeeId,
                t.clockIn == null ? "" : t.clockIn.toString(),
                t.clockOut == null ? "" : t.clockOut.toString(),
                Double.toString(t.hours),
                Double.toString(t.pay)
            });
        }
        CsvFiles.write(CsvFiles.p("timeclock.csv"), "id,employeeId,clockIn,clockOut,hours,pay", rows);
    }
    static Optional<TimeEntry> openShiftFor(String empId){
        return all().stream().filter(t -> empId.equals(t.employeeId) && t.clockOut==null).findFirst();
    }

    static double sumHours(String empId){
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS

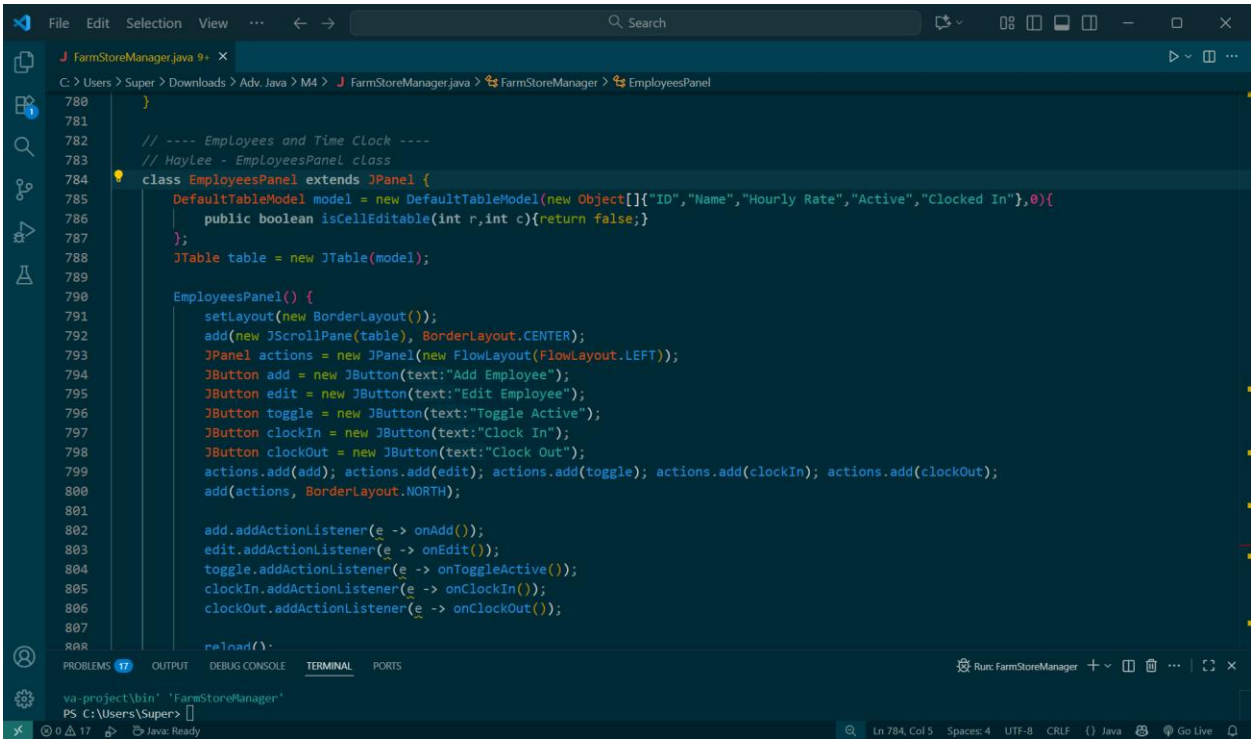
Run: FarmStoreManager

```
va-project\bin' 'FarmStoreManager'
PS C:\Users\Super>
```

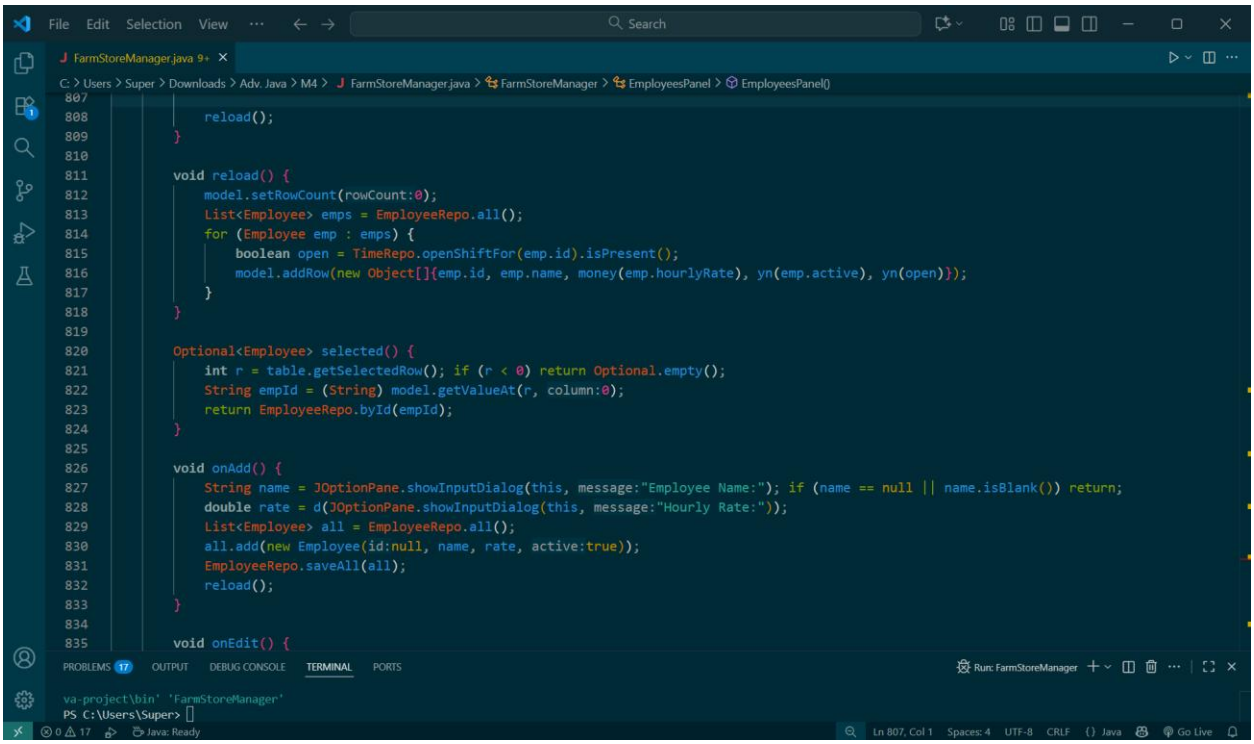


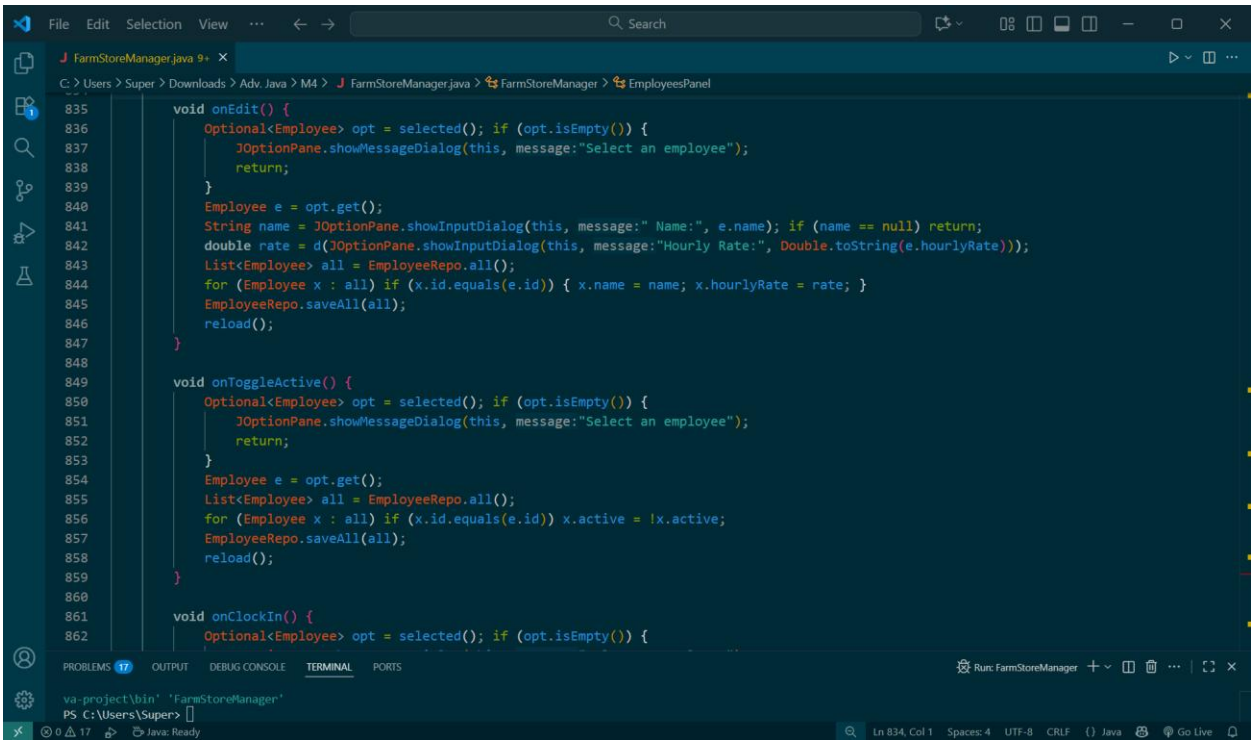
```
468         static double sumHours(String empId){
469             return all().stream().filter(t -> empId.equals(t.employeeId)).mapToDouble(t -> t.hours).sum();
470         }
471
472         static double sumPay(String empId){
473             return all().stream().filter(t -> empId.equals(t.employeeId)).mapToDouble(t -> t.pay).sum();
474         }
475     }
476
477     // parsing helpers
478     static int i(String s){ try { return Integer.parseInt(s.trim()); } catch(Exception e){ return 0; } }
479     static double d(String s){ try { return Double.parseDouble(s.trim()); } catch(Exception e){ return 0.0; } }
480     static boolean b(String s){ return "true".equalsIgnoreCase(s.trim()); }
481     static String n(String s){ return (s==null || s.isBlank())? null : s; }
482     static String nn(String s){ return s==null? "" : s; }
483
484     // sales compute
485     static void computeTotals(Sale sale){
486         double sub=0, tax=0;
487         for (SaleLine l : sale.lines){ sub += l.lineTotal; if (l.taxable) tax += l.lineTotal*TAX_RATE; }
488         sale.subTotal=sub; sale.tax=tax; sale.total=sub+tax;
489     }
490
491     // ==================== Panels ====================
492
493     // ---- Store (Inventory & Item Sales) ----
494     class StorePanel extends JPanel {
495         DefaultTableModel model = new DefaultTableModel(new Object[]{"SKU","Name","Category","Price","Qty","Taxable"}, 0){
```

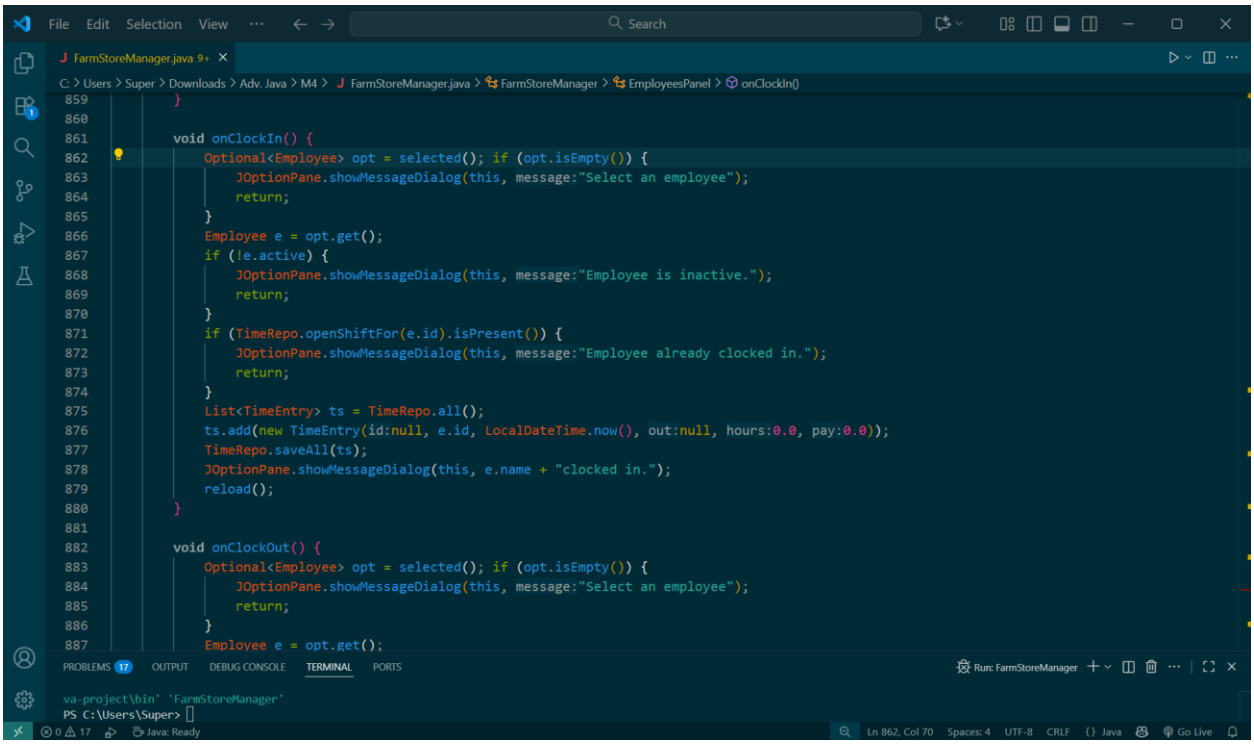
Lines 784-910:

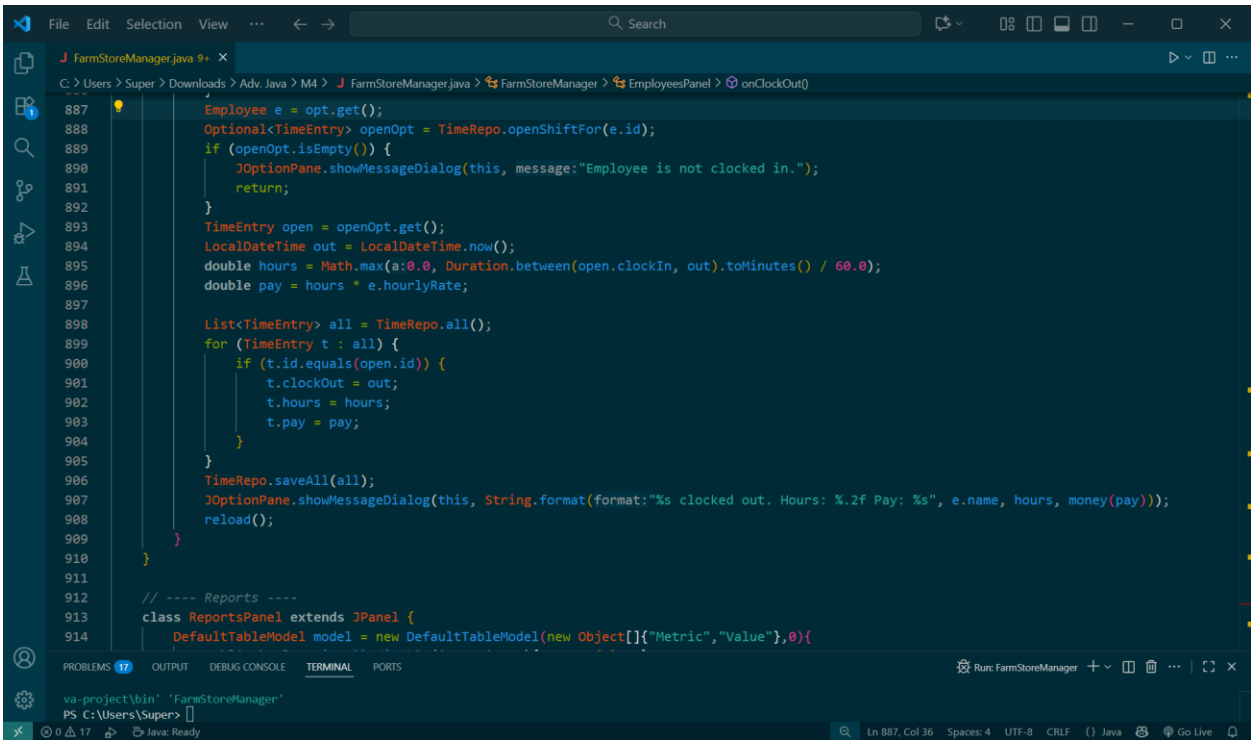


```
780     }
781
782     // ---- Employees and Time Clock ----
783     // Haylee - EmployeesPanel class
784     class EmployeesPanel extends JPanel {
785         DefaultTableModel model = new DefaultTableModel(new Object[]{"ID","Name","Hourly Rate","Active","Clocked In"},0){
786             public boolean isCellEditable(int r,int c){return false;}
787         };
788         JTable table = new JTable(model);
789
790         EmployeesPanel() {
791             setLayout(new BorderLayout());
792             add(new JScrollPane(table), BorderLayout.CENTER);
793             JPanel actions = new JPanel(new FlowLayout(FlowLayout.LEFT));
794             JButton add = new JButton(text:"Add Employee");
795             JButton edit = new JButton(text:"Edit Employee");
796             JButton toggle = new JButton(text:"Toggle Active");
797             JButton clockIn = new JButton(text:"Clock In");
798             JButton clockOut = new JButton(text:"Clock Out");
799             actions.add(add); actions.add(edit); actions.add(toggle); actions.add(clockIn); actions.add(clockOut);
800             add(actions, BorderLayout.NORTH);
801
802             add.addActionListener(e -> onAdd());
803             edit.addActionListener(e -> onEdit());
804             toggle.addActionListener(e -> onToggleActive());
805             clockIn.addActionListener(e -> onClockIn());
806             clockOut.addActionListener(e -> onClockOut());
807
808             reload();
```

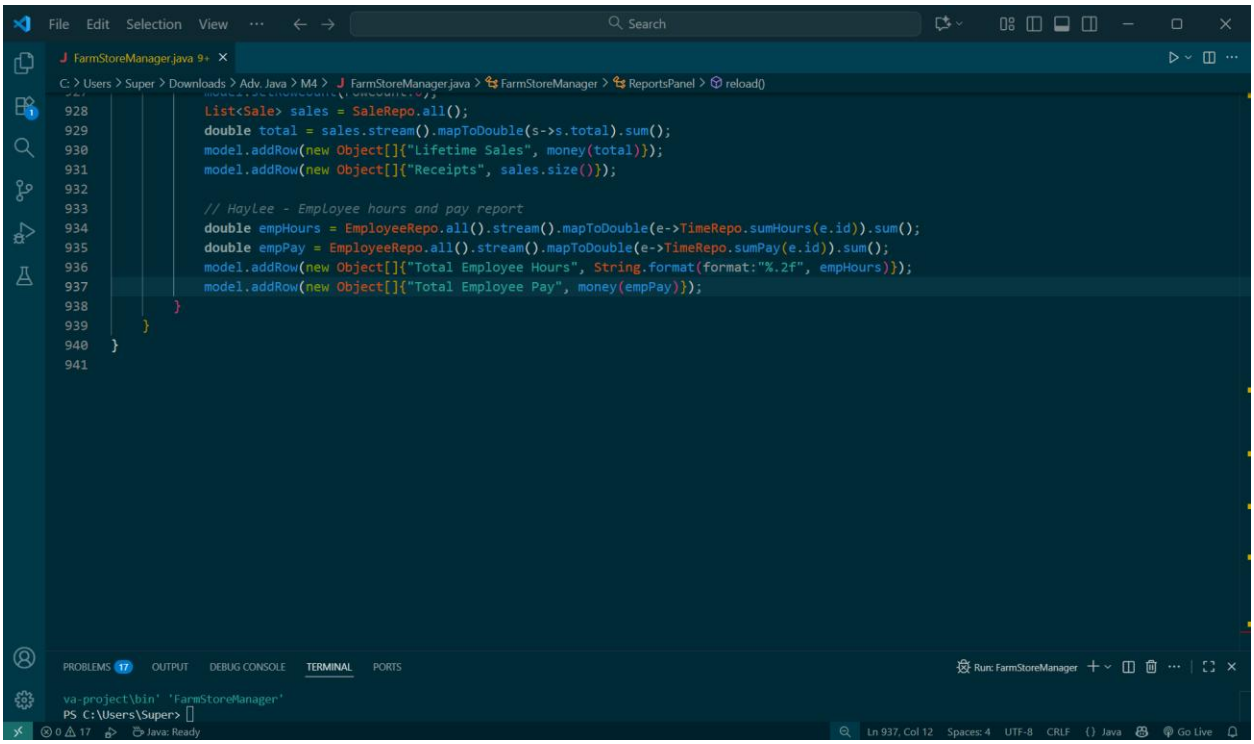

File Edit Selection View
Search
FarmStoreManager.java 9+
C: > Users > Super > Downloads > Adv. Java > M4 > FarmStoreManager.java > FarmStoreManager > EmployeesPanel > EmployeesPanel()
reload();
}
void reload() {
model.setRowCount(rowCount:0);
List<Employee> emps = EmployeeRepo.all();
for (Employee emp : emps) {
boolean open = TimeRepo.openShiftFor(emp.id).isPresent();
model.addRow(new Object[]{emp.id, emp.name, money(emp.hourlyRate), yn(emp.active), yn(open)});
}
}
Optional<Employee> selected() {
int r = table.getSelectedRow(); if (r < 0) return Optional.empty();
String empId = (String) model.getValueAt(r, column:0);
return EmployeeRepo.byId(empId);
}
void onAdd() {
String name = JOptionPane.showInputDialog(this, message:"Employee Name:"); if (name == null || name.isBlank()) return;
double rate = d(JOptionPane.showInputDialog(this, message:"Hourly Rate:"));
List<Employee> all = EmployeeRepo.all();
all.add(new Employee(id:null, name, rate, active:true));
EmployeeRepo.saveAll(all);
reload();
}
void onEdit() {
PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Run: FarmStoreManager
va-project\bin' 'FarmStoreManager'
PS C:\Users\Super>
Java: Ready
Ln 807, Col 1 Spaces: 4 UTF-8 CRLF Java Go Live


File Edit Selection View
Search
FarmStoreManager.java 9+
C: > Users > Super > Downloads > Adv. Java > M4 > FarmStoreManager.java > FarmStoreManager > EmployeesPanel
void onEdit() {
Optional<Employee> opt = selected(); if (opt.isEmpty()) {
JOptionPane.showMessageDialog(this, message:"Select an employee");
return;
}
Employee e = opt.get();
String name = JOptionPane.showInputDialog(this, message:" Name:", e.name); if (name == null) return;
double rate = d(JOptionPane.showInputDialog(this, message:"Hourly Rate:", Double.toString(e.hourlyRate)));
List<Employee> all = EmployeeRepo.all();
for (Employee x : all) if (x.id.equals(e.id)) { x.name = name; x.hourlyRate = rate; }
EmployeeRepo.saveAll(all);
reload();
}
void onToggleActive() {
Optional<Employee> opt = selected(); if (opt.isEmpty()) {
JOptionPane.showMessageDialog(this, message:"Select an employee");
return;
}
Employee e = opt.get();
List<Employee> all = EmployeeRepo.all();
for (Employee x : all) if (x.id.equals(e.id)) x.active = !x.active;
EmployeeRepo.saveAll(all);
reload();
}
void onClockIn() {
Optional<Employee> opt = selected(); if (opt.isEmpty()) {
PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Run: FarmStoreManager
va-project\bin' 'FarmStoreManager'
PS C:\Users\Super>
Java: Ready
Ln 834, Col 1 Spaces: 4 UTF-8 CRLF Java Go Live





Lines 934-937:


File Edit Selection View
Search
FarmStoreManager.java 9+
C: > Users > Super > Downloads > Adv. Java > M4 > FarmStoreManager.java > FarmStoreManager > ReportsPanel > reload()
928 List<Sale> sales = SaleRepo.all();
929 double total = sales.stream().mapToDouble(s->s.total).sum();
930 model.addRow(new Object[]{"Lifetime Sales", money(total)});
931 model.addRow(new Object[]{"Receipts", sales.size()});
932
933 // Haylee - Employee hours and pay report
934 double empHours = EmployeeRepo.all().stream().mapToDouble(e->TimeRepo.sumHours(e.id)).sum();
935 double empPay = EmployeeRepo.all().stream().mapToDouble(e->TimeRepo.sumPay(e.id)).sum();
936 model.addRow(new Object[]{"Total Employee Hours", String.format(format:"%.2f", empHours)});
937 model.addRow(new Object[]{"Total Employee Pay", money(empPay)});
938 }
939 }
940 }
941
PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Run: FarmStoreManager
va-project\bin' 'FarmStoreManager'
PS C:\Users\Super>
Java: Ready
Ln 937, Col 12 Spaces: 4 UTF-8 CRLF Java Go Live